

Synthetic Aperture Tilt-Shift from a Handheld Phone Video

ELEC 549 project report · Daniel Kuo · 23 September 2026

1. Creative goal

I wanted one photograph of a real place whose plane of focus is *tilted through the scene*, taken with nothing but an iPhone. A tilt-shift lens can do this and it is what gives the miniature, model-village look; a phone lens cannot, and app filters only paint a blur band across the picture. The goal was the real optical effect: every object blurred by its true distance from a plane chosen after the fact.

The final image looks down at an angle from an upper floor of O'Connor into the two-storey lobby. The focus plane lies along the upper floor, so the lounge furniture is sharp while the study table one floor below, the stairs and the column beside the camera fall away from the plane and blur.



Figure 1. Final image: the O'Connor lobby from an upper floor. Focus plane tilted 26° off the lens axis; average of 169 aligned video frames.

2. How it works

A big lens is an average of many small views. Light enters a lens at every point across its opening, and each point sees the scene from a slightly different position. The sensor adds all of those views together. An object at the focus distance looks the same from every point of the opening, so the views agree and it stays sharp. An object nearer or farther is seen from slightly different angles, the views disagree about where it is, and the sum smears it out. That smear is defocus blur, and it grows with the size of the opening. A phone lens is a few millimetres across, so its views barely disagree and almost nothing blurs.

A video sweep is a big lens taken one view at a time. If I move the phone across a disc about an arm's reach wide while recording, each frame is the view from one point of a lens that size. Adding the frames together gives the image that lens would have made. This is Marc Levoy's synthetic aperture idea, and a 0.5 m opening is a hundred times wider than any real lens, which is why the blur becomes strong enough to look like a miniature.

Aligning the frames is what sets the focus. Before adding, the frames have to be shifted so that some chosen part of the scene lands in the same place in every one. Whatever is aligned comes out sharp; everything else comes out blurred. The simplest version, which I built first, aligns on one small patch by template matching and shifts each whole frame by that amount. That focuses on the depth of the patch, the way SynthCam focuses on the object the user taps.

Why the blur is real. Moving the camera sideways by t makes a point at distance z shift in the image by about $f \cdot t / z$ pixels (f is the focal length in pixels). Near things shift a lot, far things a little: that is parallax. If every frame is shifted back by the amount that suits depth z_0 , points at z_0 land exactly on top of each other, but a point at another depth z is still off by $f \cdot t \cdot |1/z - 1/z_0|$. Over the whole sweep, t ranges across the disc of diameter D , so that point is spread over

$$b = D \cdot f \cdot |1/z - 1/z_0| \text{ pixels.}$$

This is the circle-of-confusion formula for a real lens with D in place of the aperture diameter. No depth map is estimated and no blur filter is applied; the blur is the frames disagreeing, by exactly the amount their parallax disagrees.

Tilting the plane. A single shift per frame can only focus on one depth, and the sharp region is then a slab facing the camera, like an ordinary lens. To focus on a tilted plane, every pixel needs a different correction, because the plane is at a different depth at every pixel. It turns out that for any flat plane, the whole per-pixel correction from the reference view to frame i is one 3×3 matrix, a homography:

$$H = K (R + t n^T / d) K^{-1}$$

Reading it from right to left: K^{-1} turns a pixel of the reference frame into a ray in space; R turns that ray by however much the phone rotated between the two frames; $t n^T / d$ adds the parallax, which is proportional to the translation t and inversely proportional to how far along that ray the plane lies, which is what the plane's normal n and distance d encode; and K turns the result back into a pixel of frame i . Warping frame i by H puts every point of the plane back where the reference view saw it, and the plane comes out sharp after averaging.

The important thing is what H does *not* depend on: the pictures. It is built only from the camera motion, the lens, and the numbers (n, d) that describe the plane. So the plane can be anywhere: tilted, cutting through empty air, running along a floor with no texture to track. Tilting the focus plane is just choosing a different n . The phone does not know it did this; one recording gives every plane.

3. Capture

The lobby was filmed from the upper-floor balcony with an iPhone at 4K, 24 fps, on the $1\times$ lens, for about 45 seconds. During the take the phone was moved across a disc roughly an arm's reach wide, spiralling out from the centre, while kept pointed at the same spot in the lobby. Only the phone's position matters; small rotations are corrected by the alignment later. Every fifth frame was extracted, giving 213 frames spread over the disc.

The scene was chosen on purpose: looking steeply down from a height at furniture on two floors, with a column very close to the camera, gives depths from a couple of metres to about fifteen in one frame. That depth range is what the effect needs (Section 7).

4. Technical steps

I wrote a small Python tool, `sa`, with one command per stage. Every stage writes into one project folder and records its parameters, so any output can be traced back to the settings that made it.

1. Extract frames. `ffmpeg` decodes the HEVC `.mov` and writes every fifth frame as a PNG. 200 frames are plenty; more only slows the next step.

2. Recover the camera positions. COLMAP, called from Python through `pycolmap`, finds thousands of SIFT features in each frame, matches them between neighbouring frames, and solves structure-from-motion: it returns the focal length, the lens distortion, the rotation R and position t of the camera for every frame, and a sparse cloud of about ten thousand 3D points it triangulated along the way. Only the sparse stage is needed, so it runs on the CPU. Out of the box COLMAP failed: its defaults assume photographs taken metres apart, and it refuses to start from a pair of views that are less than 16° apart as seen from the scene, but a hand sweep never exceeds a degree or two. Lowering that threshold to 2° (and the related triangulation limits to 0.2°) let it register all 213 frames with a mean reprojection error of 1.3 px. The focal length came out at 2658 px.

3. Choose the plane. The output is rendered from the frame nearest the middle of the sweep, and the plane is described in that camera's coordinates. In my GUI, clicking on the image sets the focus depth from the sparse points near the click and makes that pixel the pivot; two sliders then set the tilt (degrees off an ordinary lens's focus plane) and the azimuth (which way it tips), and a third the distance. The sparse points are drawn over the image, coloured green where the chosen plane would render them sharp, so the sharp band is visible before anything is rendered. A preview at reduced resolution updates in about a tenth of a second.

4. Warp and average. For each frame the tool computes H from that frame's R and t and the chosen (n, d) , undistorts the frame, warps it by H , and adds it into a floating-point accumulator, one frame at a time so that 200 frames of 4K never sit in memory together. Two details matter here. First, the average is taken in linear light: video pixels are stored through a gamma curve, and averaging those values directly makes every blurred region come out too dark, so each frame is decoded to linear intensity before adding and the result is re-encoded once at the end. Second, a weight image is accumulated alongside, counting how many frames covered each pixel, so that the edges of the picture, which not every frame reaches, are not darkened.

5. Stop down if needed. An *aperture* control keeps only the frames whose camera position lies within a fraction of the sweep's radius. The final image uses the inner 80 % (169 of 213 frames): the outermost frames are the ones whose positions COLMAP recovered least accurately, and they cost more sharpness on the plane than they added blur off it.

6. Render. A full 4K render takes a few seconds, and a whole tilt sweep can be rendered in one pass over the frames. I also ported the warp-and-average step to a WebGL shader so that it runs live in a browser, at about 3 ms per image; the demo is at <https://aperture-tilt-shift.vercel.app> and the source at <https://github.com/danielhkuo/aperture-tilt-shift>.

5. The final image in numbers

Structure-from-motion recovers shape but not absolute scale, so distances are in arbitrary units; the blur in pixels only depends on the ratio D/z .

Quantity	Value
Frames	213 extracted, 213 registered by COLMAP, 169 used (aperture 0.8)
Intrinsics	$f = 2658$ px, radial distortion $k = 0.015$, reprojection error 1.27 px
Sweep diameter	10.4 units for the full sweep, 7.8 for the frames used
Scene depth	230–640 units (5th–95th percentile of the sparse points)
Focus plane	tilt 26° , azimuth 351° , depth 284 at the pivot (upper floor by the sofa)

Predicted blur $b = D f |1/z - 1/z_0|$ at a few points: 0 px at the pivot, 1 px at the top of the stairs, 10 px at the reception desk, 33 px at the table on the lower floor (at 4K resolution).



Figure 2. Full-resolution crops, reference frame left and render right. Top: the lower-floor table (predicted 33 px) is smeared into a soft disc. Bottom: the sofa on the focus plane stays sharp; a person who walked past during the sweep survives as a translucent ghost.



Figure 3. A second scene, the Commons dining hall, from one video and two planes. Left: tilted 38.5° along the floor, so tables near and far are sharp while the banners in front and the courtyard beyond blur. Right: tilted 63.5° the other way; the sharp band now runs across the mezzanine and the floor is out of focus.

6. Validation

Before trusting real footage I rendered a synthetic city (a textured ground plane seen from above, with upright buildings) from known camera positions and ran the full pipeline on the encoded video. COLMAP recovered $f = 1001$ px against a true 1000 and registered all 60 frames; clicking three ground points gave a plane tilted 54.8° against the true 55° . A second test renders dots at known depths by direct projection, sharing no code with the homography, and measures their rendered width: it matches $D f |1/z - 1/z_0|$ within 6 %, and a template-matched shift-and-add of the same frames agrees with the pose-derived warp to within one grey level.



Figure 4. Tilt sweep of the synthetic scene from one capture, 0° to 60° , rendered in one pass over the frames.

7. What I learned

The scene decides whether the effect exists at all. The blur at a point is $D \cdot f \cdot |1/z - 1/z_0|$: it comes from the *difference* between that point's depth and the focus depth, scaled by how big the sweep is compared with the distance. A scene where everything is at nearly the same depth, a wall seen head-on or a distant view, produces almost no blur no matter how the plane is set, because the frames agree about everything. A test capture of the quad from a roof, some 50 m away, came out almost uniformly sharp for exactly this reason. The lobby works because from the balcony there are objects at two metres (the column), at five (the upper-floor furniture) and at fifteen (the study table below), all in one frame. Even so, the predicted 33 px of blur on the lower floor is visible but modest; the miniature look would be stronger with a wider sweep or a nearer subject. Looking down from a height at small objects, the classic tilt-shift viewpoint, is the right choice for optical reasons, not only for the look.

Moving people do not blur, they ghost. A real lens takes all of its views at the same instant, so a person is at one place in every view and simply blurs if they are off the plane. My aperture is assembled over 45 seconds. Someone who walks through is at a different place in each frame, and in most frames not there at all, so the average contains a faint, transparent copy of them at each position they occupied (Figure 2, bottom right, and the diners in Figure 3). This is not a bug in the alignment; it is the honest result of averaging views taken at different times. Taking the median of the frames instead of the mean drops anything that is present in fewer than half the frames, at the cost of changing the character of the blur; shooting when nobody is moving is the simpler fix.

The focus plane can go where no lens can put it. A lens's focus plane is fixed by its geometry: perpendicular to the axis, or tilted only within the limits of a tilt-shift mechanism, and always a plane the lens is physically pointed at. Here the plane is three numbers. In Figure 3 (right) it passes through the open air of the atrium and across walls with nothing on them to track, and in the lobby it runs along a floor at 26° to the line of sight. SynthCam cannot do this, because it aligns on image features and there has to be something at the focus distance to tap. Computing the warp from camera pose removes that requirement; the price is that the pose has to be recovered first, which is the slow and fragile step. Once it is, the render is cheap enough that the whole family of images from one recording can be explored live with a slider, which turned out to be the most convincing demonstration of what a synthetic aperture is.

The alignment is only as good as the recovered camera positions. Any error in a frame's R and t misplaces that frame's contribution and softens the very plane that should be sharp. The frames at the far edge of the sweep were the least accurately placed, which is why the final image uses the inner 80 % of the sweep, and why a second take of the same lobby, on which COLMAP registered only a handful of frames, could not be used at all. For this method the reconstruction stage, not the rendering, is where the risk lives.